

Class Attributes and Methods Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-class-attributes-and-methods-lab>  <https://github.com/learn-co-curriculum/python-p3-class-attributes-and-methods-lab/issues/new>

Learning Goals

- Use class attributes and methods to write durable and powerful code.
 - Store and access song data using class attributes and methods.
- Accomplish complex programming tasks using knowledge from previous modules.

Key Vocab

- **Attribute:** variables that belong to an object.
- **Constant:** variable whose value cannot be changed.
- **Instance:** one specific working copy of a class. It is created when a class's `__init__` method is called.
- **Class:** a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Static:** an attribute or method that cannot manipulate the class or instance it belongs to.
- **Exception:** an error that occurs during the execution of a program. Exceptions can be anticipated and handled without disrupting the execution of the program.

Introduction

In this lab, we'll be dealing with a `Song` class. The `Song` class can produce individual songs. Each song has a name, an artist and a genre. We need our `Song` class to be able to keep track of the number of songs that it creates.

```
Song.count  
# => 30
```

We need our `Song` class to be able to show us all of the artists of existing songs:

```
Song.artists  
# ["Jay-Z", "Drake", "Beyonce"]
```

We need our `Song` class to be able to show us all of the genres of existing songs:

```
Song.genres  
# => ["Rap", "Pop"]
```

We also need our `Song` class to be able to keep track of the number of songs of each genre it creates.

In other words, calling:

```
Song.genre_count
```

Should return something like this;

```
{"Rap": 5, "Rock": 1, "Country": 3}
```

Lastly, we want our `Song` class to reveal to us the number of songs each artist is responsible for.

```
Song.artist_count  
# {"Beyonce": 17, "Jay-Z": 40}
```

We'll accomplish this with the use of **class attributes** and **class methods**.

Instructions

Define your `Song` class such that an individual song is initialized with a name, artist and genre.

```
ninety_nine_problems = Song("99 Problems", "Jay-Z", "Rap")

ninety_nine_problems.name
# "99 Problems"

ninety_nine_problems.artist
# "Jay-Z"

ninety_nine_problems.genre
# "Rap"
```

Create a class attribute, `count`. We will use this attribute to keep track of the number of new songs that are created from the `Song` class. Set this attribute equal to `0`.

At what point should we increment our `count` of songs? Whenever a new song is created. Your `__init__` method should call a class method `add_song_to_count()` that increments the value of `count` by one.

Next, define the following class methods:

`add_to_genres()` : adds any new genres to a class attribute `genres`, a list. This list should contain only *unique genres* — no duplicates! Think about what you'll need to do to get this method working:

- You'll need a class attribute, let's call it `genres`, that is equal to an empty list.
- When should you add genres to the array? Whenever a new song is created. Your `__init__` method should add the genre of the song being created to the `genres` list. All genres should be added to the list. Control for duplicates when you code your `add_to_genres` class method, not when you add genres to the original `genres` list. We will want to know how many songs of each genre have been created. We'll revisit that job a little later on.

`add_to_artists()` : adds any new artists to a class attribute `artists` , a list. This list should only contain unique artists, just like the `genres` class attribute. Once again, think about what you need to do to implement this behavior:

- You'll need a class attribute, `artists` , that is equal to an empty list.
- When should you add artists to this array? Whenever a new song is initialized. Your `__init__` method should add artists to the `artists` list. All artists should be added to the list. Control for duplicates when you code your `add_to_artists()` class method, not when you add artists to the original `artists` list. We will want to know how many songs each have been assigned to each artist. We'll revisit that job a little later on when we write our `add_to_artist_count()` method.

`add_to_genre_count()` : adds to a class attribute `genre_count` , a dictionary in which the keys are the names of each genre. Each genre name key should point to a value that is the number of songs that have that genre.

```
Song.genre_count  
# {"Rap": 5, "Rock": 1, "Country": 3}
```

This manner of displaying numerical data is called a [histogram](https://en.wikipedia.org/wiki/Histogram). How will you create your histogram? There are a few ways!

- You can need to iterate over the `genres` list and populate a dictionary with the key/value pairs. You will need to check to see if the hash already contains a key of a particular genre. If so, increment the value of that key by one, otherwise, create a new key/value pair.

`add_to_artist_count()` : creates a histogram similar to the one above, but for artists rather than genres.

Resources

- [Python Documentation](https://docs.python.org/3/)
- [Classes - Python](https://docs.python.org/3/)
- [Python Class Attributes: An Overly Thorough Guide - Toptal](https://www.toptal.com/python/python-class-attributes-an-overly-thorough-guide)
- [Python's Instance, Class, and Static Methods Demystified - Real Python](https://realpython.com/instance-class-and-static-methods-demystified/)

- [The Factory Method Pattern and Its Implementation in Python - Real Python](https://realpython.com/factory-method-python/) ➞ [\(https://realpython.com/factory-method-python/\)](https://realpython.com/factory-method-python/)

This tool needs to be loaded in a new browser window

Load Class Attributes and Methods Lab (CodeGrade) in a new window